



**MINNESOTA STATE UNIVERSITY, MANKATO  
DEPARTMENT OF COMPUTER INFORMATION SCIENCE  
COMPUTER SCIENCE PROGRAM**

**TEST PLAN DOCUMENT**

**THE MavPASS SCHEDULING PROJECT**

**Clients:**

Dr. Laura Jacobi  
MavPASS Faculty Liaison

Lina Wang  
MavPASS Coordinator

**THE MavPASS SCHEDULING TEAM**

Madeline Ellingson  
Software Engineer

Hiruy Gebregiorgis  
Test Engineer

Justin Engels  
Database Engineer

Marilyn Croffie  
Team Lead

**FACULTY COACH**

Dr. Lin Chase

## TABLE OF CONTENTS

|                                              |           |
|----------------------------------------------|-----------|
| <b>Title Page.....</b>                       | <b>1</b>  |
| <b>Table of Contents .....</b>               | <b>2</b>  |
| <b>Introduction .....</b>                    | <b>3</b>  |
| Project Overview .....                       | 3         |
| Purpose.....                                 | 3         |
| <b>Scope.....</b>                            | <b>3</b>  |
| In Scope.....                                | 3         |
| Out of Scope .....                           | 3         |
| <b>Testing Strategy.....</b>                 | <b>4</b>  |
| Testing Cycles.....                          | 4         |
| Alpha Testing .....                          | 4         |
| Beta Testing.....                            | 4         |
| Levels of Testing.....                       | 4         |
| Unit Testing.....                            | 4         |
| Integration Testing .....                    | 4         |
| Testing Criteria .....                       | 5         |
| Entry Criteria .....                         | 5         |
| Suspension Criteria .....                    | 5         |
| Resumption Criteria.....                     | 5         |
| Exit Criteria.....                           | 5         |
| Test Deliverables .....                      | 5         |
| <b>Execution Strategy .....</b>              | <b>6</b>  |
| Bug Severity & Priority List .....           | 6         |
| Test Metrics.....                            | 6         |
| Defect Tracking & Reporting.....             | 7         |
| <b>Dependencies.....</b>                     | <b>8</b>  |
| <b>Testing Tools &amp; Environment .....</b> | <b>8</b>  |
| <b>Test Management .....</b>                 | <b>9</b>  |
| Roles & Responsibilities .....               | 9         |
| Test Schedule .....                          | 9         |
| Risk & Mitigation .....                      | 9         |
| <b>Revision History .....</b>                | <b>10</b> |
| <b>Approvals.....</b>                        | <b>10</b> |

## INTRODUCTION

### Project Overview

The MavPASS Scheduling project seeks to build a cross-platform website app that will automate (or at least semi-automate) the scheduling process of assigning MavPASS leaders to rooms based on the availabilities of both parties to not only save time on the previous labor-intensive and expensive scheduling process but also maximize the produced schedule's quality.

### Purpose

This Test Plan describes the testing approach and overall framework that will be used to ensure testing activities are effectively implemented in the MavPASS Scheduling Project. The document introduces:

- **Test Strategy:** rules the test will be based on (objectives, assumptions) and description of the process to set up a valid test (e.g.: entry/exit criteria, creation of test cases, specific tasks to perform, scheduling, data strategy).
- **Execution Strategy:** describes how the test will be performed and process to identify and report defects, and to fix and implement fixes.
- **Test Management:** process to handle the logistics of the test and all the events that come up during execution (e.g.: communications, escalation procedures, risks, and mitigation, etc.)

## SCOPE

### In Scope

These refer to the functions within the renegotiated scope of the project that must be tested.

- Availability Matching functions: checks the availability of leaders, rooms, and preferred times for students and creates a feasible schedule.
- GetAllCourses: connects to the database and fetches all courses.
- GetCourseById: finds a course by ID and returns it from the database.
- GetCourseByField: finds all courses associated with a particular field and value.
- GetCoursesByLeaderId: connects to the database and fetches all courses supported by a particular leader.
- GetAllLeaders: connects to the database and fetches all leaders.
- GetLeaderById: finds a Leader by ID and returns it from the database.
- GetLeaderByField: finds all leaders associated with a particular field and value.
- GetLeaderByCourseId: finds all leaders associated with a particular course.
- GetAllRooms: connects to the database and fetches all MavPASS Rooms.
- GetroomById: finds a Room by ID and returns it from the database.

### Out of Scope

These refer to the features that ought to be tested but not within the current phase of the project due to renegotiated scope. These features are currently out of scope.

- UI

## TESTING STRATEGY

### TESTING CYCLES

#### Alpha Testing

Alpha testing is an advanced version of unit testing. Unit testing is building applications or programs off of testing first instead of the traditional role of first development and then testing. Alpha testing is the late-stage version of unit testing meant to be executed after finalized development. This testing is carried out by an internal testing team who while not part of the initial project is some internal group and are usually paid professionals whose job is testing. As such, this is the earliest late-stage testing earning it the Alpha testing name.

#### Beta Testing

Also referred to as user testing or customer validation, beta testing aims to ensure that end users are satisfied with a software product before you make it generally available. It is one of the final steps in your software development lifecycle (SDLC) before a product goes live.

### LEVELS OF TESTING

| Testing Level              | Description                                                                                                                                                                                                                                                                     | Entry Criteria                                                   | Exit Criteria                                                                                                                                                                                                                                                                       |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit Testing               | A unit is the smallest testable portion of an application that can be compiled, loaded, and executed. This helps to test each module separately and determine if components are fulfilling functionalities or not.                                                              | Development is completed                                         | <ol style="list-style-type: none"><li>1. All Unit tests are passed.</li><li>2. Unit test providing pre-agreed code coverage.</li></ol>                                                                                                                                              |
| Integration Testing        | Different software modules are combined and tested as a group to make the integrated system ready for testing. This testing checks the data flow from one module to another.                                                                                                    | Exit criteria of Unit Testing level have been met.               | <ol style="list-style-type: none"><li>1. All integration test cases are executed.</li><li>2. Integration test cases are either passed or for failed test cases defects are logged and agreed to take the code for next level of testing with known defects.</li></ol>               |
| System Integration Testing | It is performed on integrated systems and allows checking the system's compliance as per requirements. It involves the testing of performance, reliability, and security. This is usually the final test in the process to verify if the system meets the specification or not. | Exit criteria of Integration Testing level have been met.        | <ol style="list-style-type: none"><li>1. All System Integration test cases are executed.</li><li>2. System Integration test cases are either passed or for failed test cases defects are logged and agreed to take the code for next level of testing with known defects.</li></ol> |
| User Acceptance Testing    | This is done by the end-user or the stockholder. It is a test conducted to find if the requirements of a specification are met according to the delivery.                                                                                                                       | Exit criteria of System Integration Testing level have been met. | <ol style="list-style-type: none"><li>1. All UAT test cases are executed.</li><li>2. UAT test cases are either passed or for failed test cases defects are logged and agreed to take the code into production with known defects.</li></ol>                                         |

## TESTING CRITERIA

### Entry Criteria

The entry criteria refer to the desired conditions in order to start test execution. The following are flexible benchmarks. If they are not met, the test team will assess the risk, identify mitigation actions, and provide a recommendation.

- All test hardware platforms must have been successfully installed, configured, and functioning properly.
- All the necessary documentation, design, and requirements information should be available that will allow testers to operate the system and judge the correct behavior.
- All the standard software tools including the testing tools must have been successfully installed and functioning properly.
- Proper test data is available.
- The test environment such as lab, hardware, software, and system administration support should be ready.

### Suspension Criteria

The suspension criteria refer to conditions that make it impossible to continue testing due to the defects.

- The build contains many serious defects which seriously limit testing progress.
- Significant change in requirements suggested by the client
- Software/Hardware problems
- Assigned resources are not available when needed by the test team.

### Resumption Criteria

Testing will only resume when the problem(s) that caused the suspension have been resolved.

### Exit Criteria

The exit criteria are the desirable conditions that need to be met in order to proceed with the implementation. The following are flexible benchmarks. If they are not met, the test team will assess the risk, identify mitigation actions, and provide a recommendation.

- A certain level of requirements coverage has been achieved.
- No high-priority or severe bugs are left outstanding.
- All critical and high-severity defects have been fixed and fully tested.
- 90% of medium-severity defects have been closed
- Test environment cleanup completed.

## TEST DELIVERABLES

Test Deliverables refer to a list of all the documents, tools, and other components that have to be developed and maintained in support of the testing effort. There are different test deliverables at every phase of the software development lifecycle.

Test deliverables provided **before** testing:

- Test plans document.

Test deliverables provided **during** the testing:

- Test Scripts
- Test Data
- Error logs and execution logs.

Test deliverables provided **after** the testing cycles are over:

- Test Results/reports
- Defect Report

## EXECUTION STRATEGY

### BUG SEVERITY & PRIORITY LIST

Bug Severity and Priority fields are both very important for categorizing bugs and prioritizing if and when the bugs will be fixed. The bug Severity and Priority levels will be defined as outlined below.

#### Severity List

**Severity** is a measure of how great the impact is on the user's ability to complete the documented actions within the system.

| Severity ID | Severity level | Severity Description                                                                                                                                                                                                                                                                                                                                                       |
|-------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1           | Critical       | The module/product crashes or the bug causes nonrecoverable conditions. Examples of bugs here are: System crashes, database or file corruption, potential data loss or program hangs requiring reboot.                                                                                                                                                                     |
| 2           | High           | Major system component unusable due to failure or incorrect functionality. Bugs here cause serious problems such as a lack of functionality, or insufficient or unclear error messages that can have a major impact on the user, prevent other areas of the app from being tested, etc. These bugs can have a workaround, but the workaround is inconvenient or difficult. |
| 3           | Medium         | Incorrect functionality of component or process. There is a simple workaround for the bugs here                                                                                                                                                                                                                                                                            |
| 4           | Minor          | Documentation errors or signed off severity 3 bugs.                                                                                                                                                                                                                                                                                                                        |
| 5           | Cosmetic       | There is an insufficient or unclear error message that has no impact on product use                                                                                                                                                                                                                                                                                        |

#### Priority List

| Priority ID | Priority Level | Priority Description                                                                                                                                                                                 |
|-------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1           | Must Fix       | This bug must be fixed immediately; the product cannot ship with this bug.                                                                                                                           |
| 2           | Should Fix     | These are important problems that should be fixed as soon as possible.                                                                                                                               |
| 3           | Fix Later      | The problem should be fixed within the time available. If the bug does not delay shipping date, then fix it.                                                                                         |
| 4           | Low Priority   | It is not important (at this time) that these bugs be addressed. Fix these bugs after all other bugs have been fixed. Enhancements/ Good to have features incorporated are out of the current scope. |

### TEST METRICS

Test metrics to measure the progress and level of success of the test will be developed and shared with the client for approval. Below are some of the metrics:

| Report                              | Description                                                                                                                                                                         | Frequency                 |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Daily execution status              | <ul style="list-style-type: none"><li>To report on Pass, Fail, Total defects.</li><li>Highlight critical defects</li></ul>                                                          | Daily                     |
| Test preparation & Execution Status | <ul style="list-style-type: none"><li>To report on % complete, %Work In Progress, %Pass, % Fail</li><li>To report on defect severity-wise status i.e., open, closed, etc.</li></ul> | Weekly                    |
| Project Status report               | Project driven reporting                                                                                                                                                            | Quarterly (of a semester) |

## DEFECT TRACKING & REPORTING

The defect-tracking process will be observed using the following steps:

- The tester will report defects and assign an initial severity and status.
- The Testing Lead will review the severity of the defects and facilitate with the technical team the fix and its implementation.
- The team will attempt to fix the defects and modify the status as the defect progresses through the cycle.
- The tester will retest.
- If the defects have been fixed, the solution will be approved and the defect report will be closed, otherwise, the team will reattempt fixing the defects.

## TEST DEFECT LOG

Defects found during test execution should be recorded manually. The Test Defect Log is a tool for recording, analyzing, tracking, and closing defects.

Supply the appropriate level of detail when recording a defect. Simply put, supply enough information for the developer to find, recreate and repair the defect. Specify the screen, field, behavior, and actual result that occurred. Include a screen capture, when possible.

### Information:

1. **Project Name:** The name of the project.
2. **Summary:** Succinct description of the problem, typically a single sentence.
3. **Reported by:** The name of the person who discovered the defect.
4. **Date Reported:** The date the defect was reported.
5. **Recorder:** The person responsible for submitting, tracking, and closing defects in the Test Defect Log.
6. **Test Category:** Nature of the testing performed. Place an **X** in the space provided beside the appropriate category.
7. **Defect ID:** A unique, sequential number used to identify the defect.
8. **Test Cycle:** The end-to-end execution of tests, per phase. Multiple tests may be executed during each phase. Tests may be repeated in subsequent cycles.
9. **Test Case ID:** The reference number of the specific test case or test script.
10. **Description:** A more detailed explanation of what actually happened.

The following three fields should be included in the description:

- Steps to reproduce – The shortest sequence of steps you’ve found that shows the defect
  - Expected behavior – What should happen
  - Observed behavior – What actually happened
11. **Severity:** The classification of defects within a system examines both the severity and priority of the defect.
  12. **Defect Codes:** CM=Configuration Management, CO=Coding, CS=Coding Standards, DC=Documentation Content, DE=Design, DP=Documentation Presentation, IA=Integration Agreement, PE=Performance, RE=Requirements, TR=Traceability, TP=Test Plan.
  13. **Status:** A state through which a defect passes from identification to closure.
  14. **Resolution:** The resolution is the solution identified for the defect. There are four possible solutions: duplicate, enhancement request, fixed, fixed indirectly, and functions as designed. This field should further be used to record comments on the nature of the resolution.
  15. **Date Resolved:** The date the defect is resolved

[illegible]

The functions under test have data dependencies. So, to isolate the behavior of the function, we want to replace the real data with mocks — our cleaned sample data — that simulate the behavior of the real data.

| Resources | Descriptions                                                                                                                                          |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Server    | Access the web application under test via GitHub<br>This includes a separate web server, database server, and application server                      |
| Test tool | The testing tool is to automate the testing, simulate the user operation, generate the test results<br>A testing tool under consideration is Selenium |
| Network   | You need a Network including LAN and Internet to simulate the real business and user environment                                                      |
| Computer  | The PC which users often use to connect to the web server                                                                                             |
| IDE       | Install Microsoft Visual Studio 2022                                                                                                                  |



## TEST MANAGEMENT

### ROLES & RESPONSIBILITIES

The Test Engineer, Hiruy Gebregiorgis, will serve as the lead for testing on this project. However, all other team members (Madeline Ellingson, Justin Engels & Marilyn Croffie) are equally responsible for testing. The following are some expected responsibilities:

- Understand test requirements
- Develop test conditions, test cases, expected results, and execution scripts.
- Perform execution and validation.
- Identify, document, and prioritize defects according to the guidance provided by the Test lead.
- Re-test after source code modifications have been made according to the schedule.
- Prepare testing metrics and provide regular status.

### TEST SCHEDULE

| Task               | Start Date                     | End Date                       | Description                                          |
|--------------------|--------------------------------|--------------------------------|------------------------------------------------------|
| Alpha Unit Testing | October 14 <sup>th</sup> 2022  | November 9 <sup>th</sup> 2022  | Testing the base functionalities in the alpha stage  |
| Beta Unit Testing  | November 12 <sup>th</sup> 2022 | December 16 <sup>th</sup> 2022 | Retesting the base functionalities in the beta stage |

### RISK

| Risk                                                       | Probability | Impact | Mitigation Plan                                                                                                                                                                                                  |
|------------------------------------------------------------|-------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SCHEDULE</b><br>Tight test scheduling                   | High        | High   | Set test priority, run all unit tests, and run high-priority tests to meet the immediate deadline. Run all other tests after the deadline                                                                        |
| <b>RESOURCES</b><br>Resources onboarding too late          | Medium      | High   | Work with whatever resources are currently available until full access to resources has been received                                                                                                            |
| <b>DEFECTS</b><br>Defects discovered at a late stage       | Medium      | High   | Set bug severity, and try refactoring code with defect(s) if critical, high, or medium. Document and track other defects.                                                                                        |
| <b>SCOPE</b><br>Scope not completely defined               | Medium      | Medium | Meet with team faculty coach and/or subject matter experts to redefine the testing scope                                                                                                                         |
| <b>LEARNING</b><br>Gaps in domain knowledge                | Medium      | High   | Use online resources to learn and/or reach out to faculty coach                                                                                                                                                  |
| <b>OTHER ISSUES</b><br>Delayed testing due to other issues | Medium      | High   | Implement unit testing to make sure minimum requirements are met. Implement high-priority tests as time would allow and properly document all other tests to be run such that the next team can run those tests. |

## REVISION HISTORY

| Version | Date       | Author(s)                                                                       | Description of Change |
|---------|------------|---------------------------------------------------------------------------------|-----------------------|
| 1.0     | 10/10/2022 | Madeline Ellingson,<br>Marilyn Croffie,<br>Hiruy Gebregiorgis,<br>Justin Engels | First Draft           |
| 2.0     | 12/03/2022 | Madeline Ellingson,<br>Marilyn Croffie,<br>Hiruy Gebregiorgis,<br>Justin Engels | Final draft           |

## APPROVALS

The undersigned acknowledge that they have reviewed the Test Plan proposed by The MavPASS Scheduling Team and agree with the information presented within this document. Changes to this plan will be coordinated with and approved by, the undersigned, or their designated representatives.

| Approver/Reviewer | Role          | Date of Approval/Reviewal |
|-------------------|---------------|---------------------------|
| Dr. Laura Jacobi  | Client        |                           |
| Lina Wang         | Client        |                           |
| Lin Chase         | Faculty Coach |                           |